WEEK 01 - LOGIC

CLASS 04 - FUNCTIONS

EX10 - Ecuacion del cohete como funcion

Multiples parametros, parametros por defecto y reutilizacion de una funcion de ingenieria

1. Donde estamos

Seguimos en week_01_logic -> class_04_functions. No cambiamos de week ni de clase. EX09 introdujo def, parametros, argumentos, return, scope y reutilizacion. EX10 amplia esas ideas a una funcion con varios parametros y una constante fisica con valor por defecto.

2. Modelo fisico: ecuacion de Tsiolkovsky

Para un cohete ideal, despreciando perdidas gravitatorias, aerodinamicas y de guiado, la capacidad ideal de delta-v viene dada por:

Delta-v = Isp * g0 * ln(m0 / mf)

Isp: impulso especifico [s]
g0: gravedad estandar [m/s^2]
m0: masa inicial [kg]
mf: masa final [kg]

3. De ecuacion a funcion

La funcion debe actuar como una caja negra: recibe los datos fisicos necesarios, valida que tengan sentido, calcula el cociente de masas y devuelve el delta-v. El programa principal no necesita repetir la formula.

```
def calculate_rocket_delta_v(isp, initial_mass, final_mass, g0=9.80665):
    ...
    return delta_v
```

4. Nuevo concepto: parametro por defecto

En EX02 tratamos g0 como una constante conocida. En una funcion reutilizable podemos convertirla en un parametro con valor por defecto. Asi, la llamada normal no necesita repetir 9.80665 cada vez, pero sigue siendo posible sustituirlo si queremos experimentar.

```
delta_v = calculate_rocket_delta_v(250, 500000, 50000)

# Usa g0 = 9.80665 automaticamente

delta_v = calculate_rocket_delta_v(250, 500000, 50000, g0=9.81)

# Sobrescribe el valor por defecto
```

Regla sintactica importante

Los parametros sin valor por defecto deben escribirse antes de los parametros con valor por defecto.

```
# Correcto
def f(a, b, c=10):
    ...

# Incorrecto
def f(a, b=10, c):
    ...
```

5. Parametros vs argumentos posicionales y keyword arguments

Puedes llamar una funcion por posicion o indicando el nombre del parametro. Las llamadas con nombre son especialmente utiles cuando hay varios parametros fisicos porque hacen el codigo mas legible.

```
# Posicional
calculate_rocket_delta_v(250, 500000, 50000)

# Con nombres
calculate_rocket_delta_v(
    isp=250,
    initial_mass=500000,
    final_mass=50000
)
```

6. Validaciones

Para este ejercicio mantendremos la decision que ya tomamos: Isp = 0 se permite y produce delta-v = 0. Las masas deben ser estrictamente positivas y la masa inicial no puede ser menor que la final. Si m0 = mf, el logaritmo vale 0 y el delta-v tambien.

Condicion	Interpretacion
isp < 0	Invalido
initial_mass <= 0	Invalido
final_mass <= 0	Invalido
initial_mass < final_mass	Invalido
initial_mass == final_mass	Valido: delta-v = 0

Como en EX09, puedes usar return None para terminar la funcion ante una entrada invalida. No uses sys.exit() dentro de una funcion reutilizable.

7. Algoritmo esperado

```
INPUTS
  isp
  initial_mass
  final_mass
  g0 = 9.80665 (default)

VALIDATE INPUTS
  invalid? -> return None

CALCULATE
  mass_ratio = initial_mass / final_mass
  delta_v = isp * g0 * ln(mass_ratio)
```

```
RETURN
  delta_v
```

8. Que NO quiero todavia

No uses clases, archivos externos de constantes ni excepciones con raise. Tampoco conviertas todavia la funcion en un modulo separado. El objetivo es practicar una funcion con varios parametros y un parametro por defecto.

9. Casos de prueba

Caso	Valores	Esperado
Nominal	Isp=250, m0=500000, mf=50000	Delta-v positivo
Sin impulso	Isp=0	0 m/s
Sin consumo	m0=mf	0 m/s
Masa invalida	mf=0	None
Relacion invalida	m0<mf	None
Isp invalido	Isp<0	None

10. Reutilizacion

Cuando la funcion funcione, llama varias veces sin repetir la formula. Prueba tambien una llamada con argumentos posicionales y otra con keyword arguments.

```
dv1 = calculate_rocket_delta_v(250, 500000, 50000)
dv2 = calculate_rocket_delta_v(isp=320, initial_mass=120000, final_mass=30000)
dv3 = calculate_rocket_delta_v(300, 100000, 100000)
```

11. Estructura del archivo

```
# Imports
from math import log

# Functions
def calculate_rocket_delta_v(...):
    ...

# Main inputs / calls
isp = ...
initial_mass = ...
final_mass = ...

delta_v = calculate_rocket_delta_v(...)

# Output
if delta_v is not None:
    print(...)
```

12. Learning Tracker

Concepto	EX10
Funcion con varios parametros	SI
Parametro por defecto	SI

Argumentos posicionales	SI
Keyword arguments	SI
return None como guard clause	SI
Variables derivadas dentro de la funcion	SI
Reutilizacion de una funcion de ingenieria	SI

Objetivo final: que el programa principal pueda pedir un delta-v mediante una llamada clara y reutilizable, sin conocer los detalles internos del calculo.